Recognition of Tokens

The regular definitions for tokens are as follows:

if → if

then → then

else → else

relop → < | <= | = | <> | > | >=

id → letter (letter|digit)*

num → $digit^+$ (.digit+)? (E(+|-)?$digit^+$)?

delim → blank | tab | newline

ws → $delim^+$

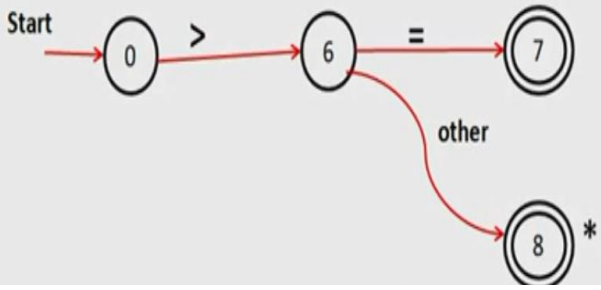
Attribute Values

Regular Expression	Token	Attribute value
ws	-	-
if	if	-
then	then	-
else	else	-
id	id	ptr to table entry
num	num	ptr to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

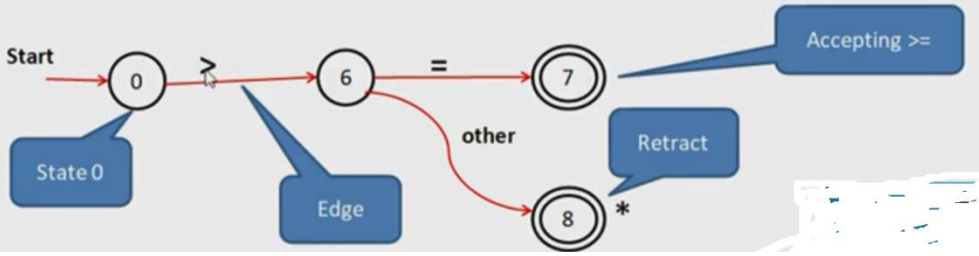
- **Transition diagram is a special kind of flowchart for language analysis.**
- **In transition diagram the boxes of flowchart are drawn as circle and called as states.**
- **States are connected by arrows called as edges.**
- **The label or weight on edge indicates the input character that can appear after that state.**

Transition Diagrams

- Lexical analysis uses transition diagram to keep track of information about characters that are seen as the forward pointer scans the input
- Transition diagrams are also called finite automata

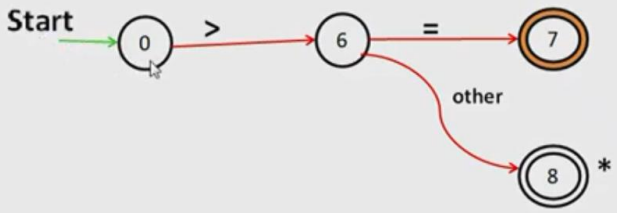


- Positions in a transition diagram are drawn as circles and are called states.
- The states are connected by arrows, called edges.
- A double circle indicated an accepting state, a state in which a token is found.
- a* indicates that input retraction must take place



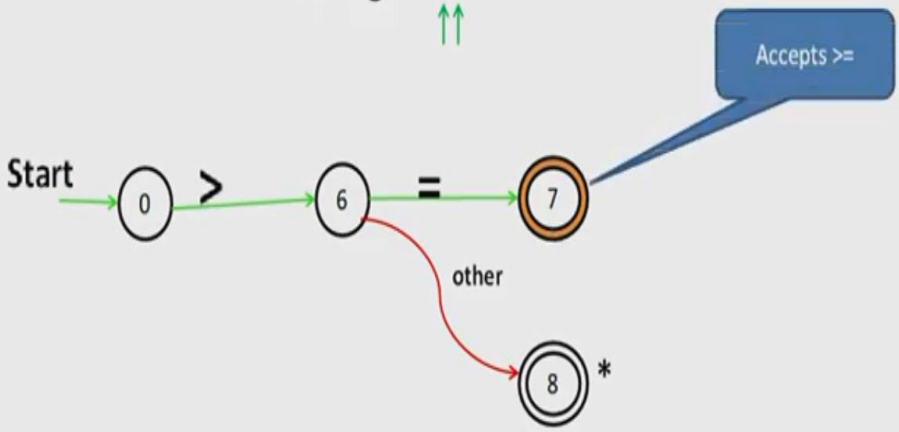
Transition Diagram

For eg: >=



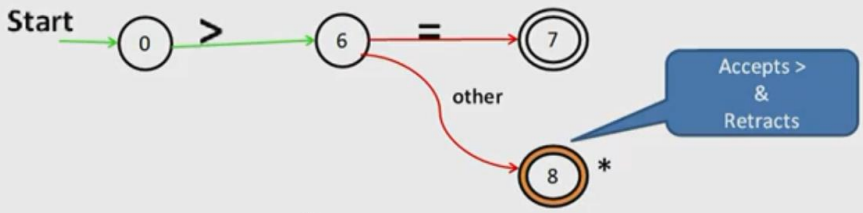
Transition Diagram

For eg: >=

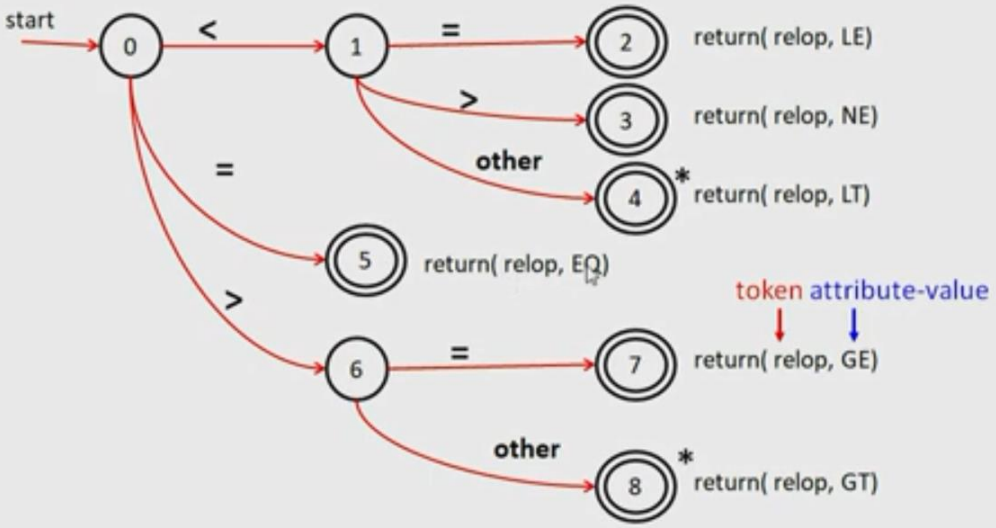


Transition Diagram

For e.g.: count>40



Transition Diagram for Relational Operators



Transition Diagram for Identifiers and Keywords



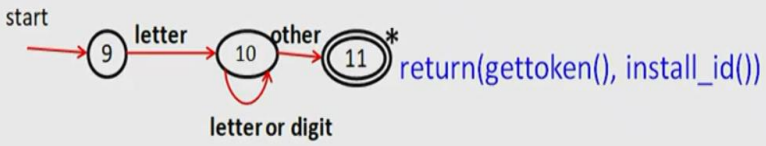
Transition Diagram for Identifiers and Keywords



gettoken( ): returns token (**id, if, then**,…) if it looks the symbol table

install_id( ): return 0 if keyword or a pointer to the symbol table entry if **id**

Transition Diagram for Identifiers and Keywords



Input String	gettoken()	install_id()
for		
if		
else		
count		
num4		

Pause the video and write

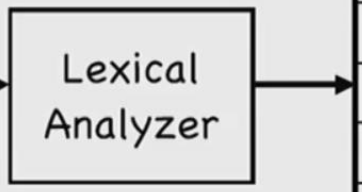
the values returned by the two functions in the table

Ex: a20

Example

```
if x <= 10
then
    x
else
    y
```

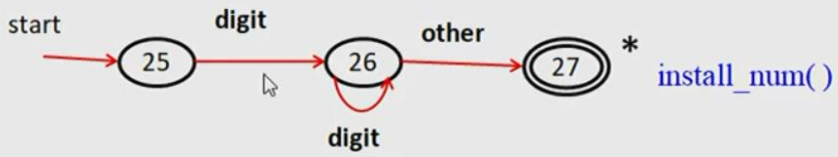
Source program



Token	Attribute
IF	NULL
ID	Ptr to x in ST
RELOP	LE
NUM	Ptr to 10 in ST
THEN	NULL
ID	Ptr to x in ST
ELSE	NULL
ID	Ptr to y in ST

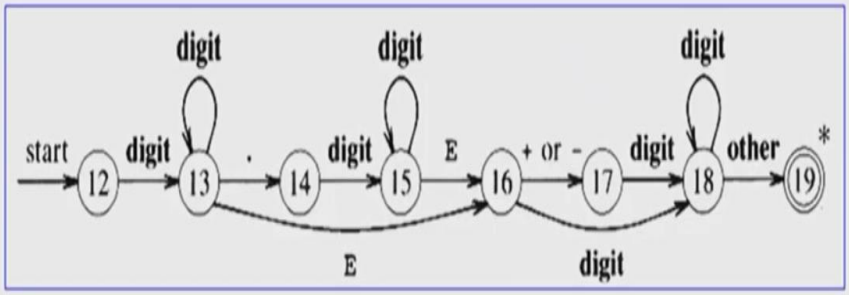
Tokens and Attributes

Transition Diagram for Unsigned Integer Numeric Cons



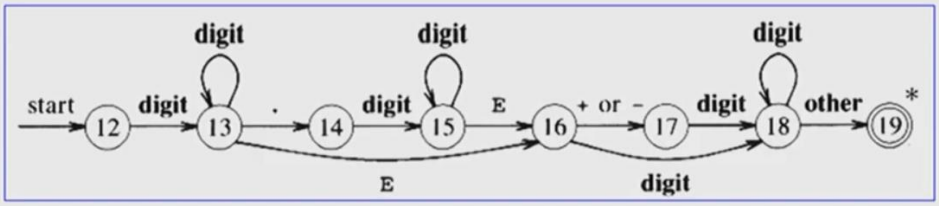
- It accepts the unsigned integer numeric constants
- Example- 314, 9, 41267 ……
- Install_num(): Enters the number in a literal table and returns a pointer to that entry

Transition Diagram for Unsigned Numbers



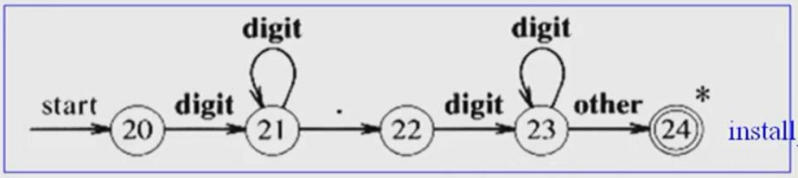
e.g. 12.3E4, 10E15

Transition Diagram for Unsigned Numbers



install_num( )

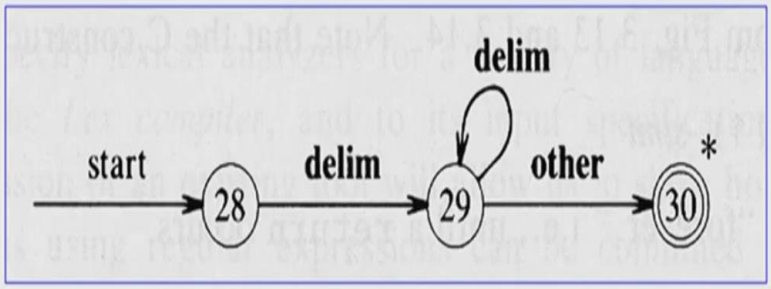
e.g. 12.3E4, 10E15



install_num( )

e.g. 12.3, 10.15

Transition Diagram for whitespace



- It accepts the whitespaces (newline, tab space and space)
- Returns the token as ws

Regular expression of identifier

letter → A|B|- - -|Z|a|b|c|- - -|z

digit → 0|1|2|- - -|9

∴ id → letter (letter | digit)*

Regular expression for unsigned number

Like, 10·58E-15

digit → 0|1|2|- - -|9

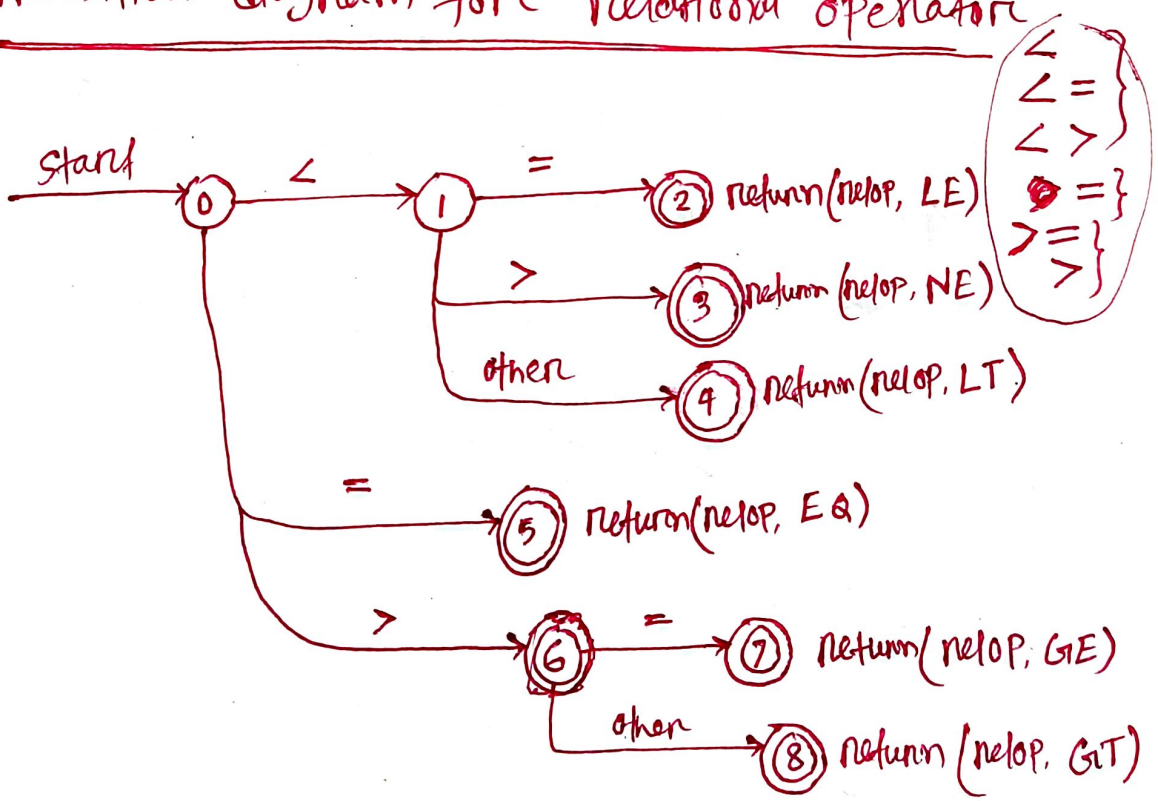
digits → digit.digit*

optional_fraction → .digits | ϵ

optional_exponent → (E(+|-|ϵ)digits) | ϵ

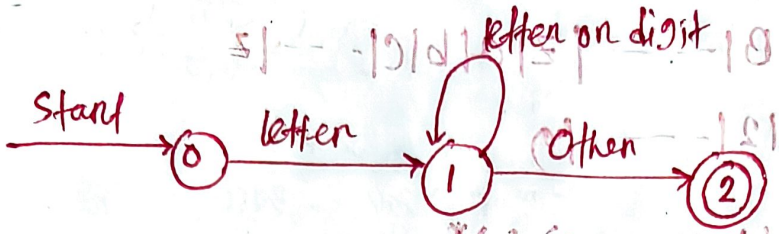
∴ num → digits optional_fraction optional_exponent

Transition diagram for relational operation



Transition diagram for identifier

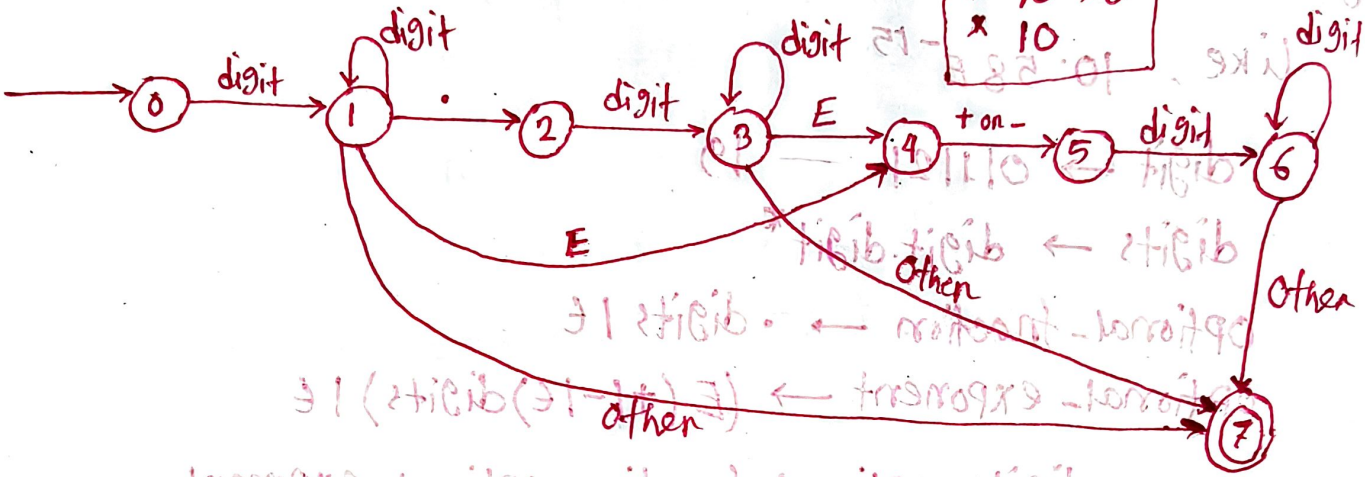
$L(L|D)^*$



Transition diagram for unsigned number

like, $10.58E^{-10}$

- $10E^{-10}$
- 10.58
- 10



$10.62E^{-10}$

digit → 0|1|2|1 —— |9

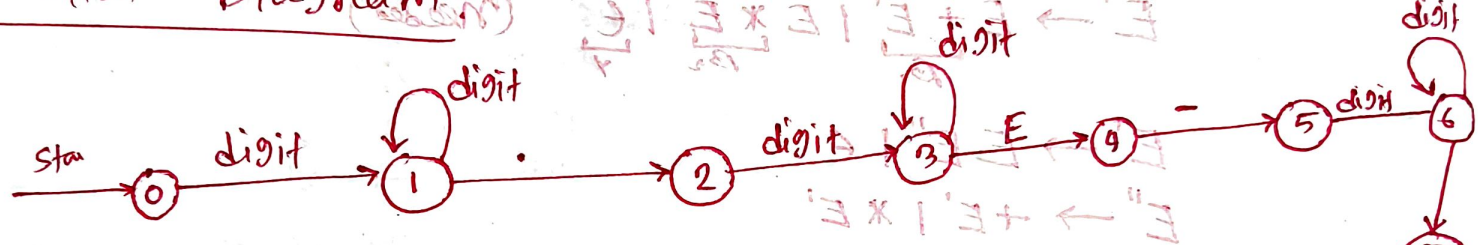
digits → digit$^+$ or digit.digit*

OP-frac → .digits | ∈

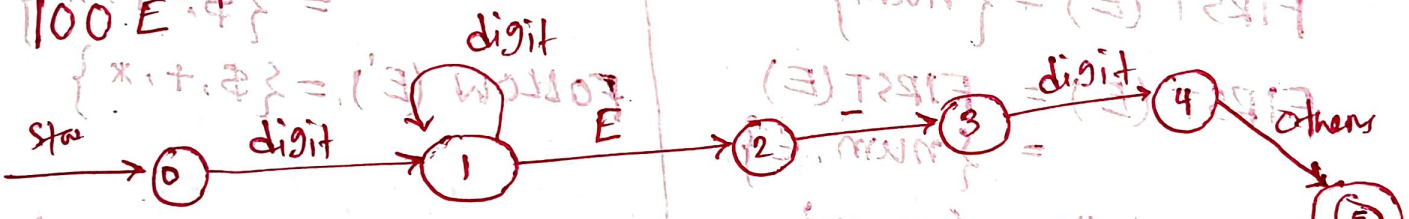
OP-exp → E (+|-|∈) digits | ∈

num → digits . OP-fract. OP-exp

Transition Diagram:



$100.E^{-9}$



105 Regular definition: D^3

CSE19 " " L^3D^2

General number → D^+